

KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ADS

MINI PROJECT REPORT

PLA-Based Rule Engine Designer

Submitted by

T THATCHINA MOORTHY

920425243109

Section B

EC2201 & EC2202 – Digital System Design and Microprocessor

October 2026

Submission of Time (5)	
Preparation (10)	
Presentation (10)	
Total	

Formatting Guide (delete this page before final submission)

- Font: Times New Roman throughout the document.
- Body text size: 12 pt. Chapter titles: 16 pt bold. Section headings: 14 pt bold.
- Line spacing: 1.5 lines throughout.
- Paragraph spacing: 6 pt space after every paragraph.
- Margins: 1 inch on top/bottom/right, 1.25 inch on the left (for binding).
- Body text alignment: Justified.
- Page numbers: bottom-center, already set up in this template's footer.
- Do not change these settings — they match the evaluation checklist.

Abstract

Programmable Logic Arrays (PLAs) are foundational reconfigurable devices in digital systems, enabling flexible implementation of combinational logic in Sum-of-Products (SOP) form. This project presents the 'PLA-Based Rule Engine Designer', a modern CAD and simulation platform developed for the EC2201 Digital System Design curriculum. The platform allows engineers and students to define Boolean rules over four variables (A, B, C, D), automatically extracts unique product terms, configures fusible link crosspoints across inverted and non-inverted rails, and applies Quine-McCluskey tabular minimization to compute minimal prime implicants. In addition, an exhaustive 16-row truth table generator, an automated rule conflict and absorption redundancy scanner, and an interactive real-time digital hardware simulator with animated current flow and LED indicators are provided. A verification suite of 10 normal operational test cases and 5 edge/fault vectors achieves a 100% pass rate. The platform bridges the gap between switching theory and physical PLD circuit implementation, offering an end-to-end environment for digital logic synthesis, verification, and academic coursework demonstration.

Table of Contents

Formatting Guide (delete this page before final submission)	2
Abstract	3
Table of Contents	4
1. Introduction	5
1.1 Real-World Problem / Use Case	5
2. Problem Statement	5
3. Implementation	5
3.1 Tools and Technologies Used	5
3.2 Implementation Steps	5
3.3 System Design / Architecture	5
4. Output / Screenshots	5
5. Conclusion and Future Scope	6
6. References	6
Submission Checklist	7

1. Introduction

Digital systems rely on Programmable Logic Devices (PLDs) to implement flexible combinational logic functions with compact silicon area. Unlike PROMs or PALs, Programmable Logic Arrays (PLAs) offer dual programmability in both AND and OR planes with product term sharing. The objective of this project is to develop a responsive, web-based PLA Rule Engine Designer that models, synthesizes, minimizes, and verifies digital logic circuits in real-time.

1.1 Real-World Problem / Use Case

Designing digital logic control units, instruction decoders, and safety interlocks often suffers from rule conflicts, redundant terms, and excessive silicon footprint. This project addresses automated logic conflict detection, crosspoint matrix generation, and hardware simulation.

2. Problem Statement

The project sets out to build an interactive PLA workbench that parses Boolean rules for variables A, B, C, D, compiles programmable AND/OR matrices, executes Quine-McCluskey minimization, detects rule conflicts, and validates circuits under active-high logic and static hazard immunity constraints.

3. Implementation

3.1 Tools and Technologies Used

Frontend: HTML5, CSS3, JavaScript, Tailwind CSS, Chart.js, HTML5 Canvas API.

Backend: Python 3.12, Flask, SQLite3, NumPy.

3.2 Implementation Steps

- Step 1 — Logic Engine & Boolean Parser: AST parsing of inputs A, B, C, D with operators AND, OR, NOT, XOR.
- Step 2 — Matrix Compilation & Minimization: Synthesized AND/OR arrays with Quine-McCluskey tabular reduction.
- Step 3 — Simulator & Verification: Live rocker switches, Canvas current-flow animations, truth table, and test suite.

3.3 System Design / Architecture

The system employs a 3-tier architecture: presentation layer (Canvas visualizer & UI), logic processing layer (Flask REST API, Quine-McCluskey engine), and persistence layer (SQLite database storing rules and test datasets).

4. Output / Screenshots

- Figure 4.1: Interactive PLA Hardware Simulator showing real-time switches, animated signal bus current flow, and LED indicators.
- Figure 4.2: Exhaustive 16-row Truth Table with active product term tracking and single-click CSV export.
- Figure 4.3: Automated Conflict Detection Scanner highlighting duplicate product terms and logic health score.

5. Conclusion and Future Scope

The PLA-Based Rule Engine Designer successfully models programmable logic arrays with high fidelity, verified by a 100% pass rate across 15 standard and edge test cases. The platform effectively demonstrates Quine-McCluskey minimization and rule conflict detection. Current constraints include a 4-variable input domain. Future extensions include multi-level logic synthesis using the Espresso heuristic algorithm, VHDL/Verilog RTL code generation, and automated FPGA bitstream export.

6. References

- [1] M. Morris Mano and Michael D. Ciletti, "Digital Design: With an Introduction to the Verilog HDL," 5th Edition, Pearson Education, 2013.
- [2] Thomas L. Floyd, "Digital Fundamentals," 11th Edition, Pearson Education, 2015.
- [3] Anna University, "EC2201 & EC2202: Digital System Design and Microprocessor Syllabus," Anna University Regulation, 2021.
- [4] IEEE Computer Society, "IEEE Standard VHDL Language Reference Manual," IEEE Std 1076-2008, 2009.

Submission Checklist

- PDF report uploaded on the Project Portal (Submit Work page)
- YouTube demo video link added (Unlisted visibility)
- GitHub repository link added, with README.md
(<https://github.com/thatchina2007-commits/PLA-Rule-Engine.git>)
- Hardcopy of the report submitted to the department, if required